

Experiment 9

Name: Paarthiv Suriya Sundaram Nagarajan

Roll No: 240701376

Design input forms that validate data (email, phone number) and display error messages using HTML, CSS, and JavaScript.

AIM

To design an input form that validates user data such as email and phone number and displays appropriate error messages using HTML, CSS, and JavaScript with Validator.js.

OBJECTIVE

- To create a user-friendly form
- To validate user input in real-time
- To display error and success messages
- To improve form usability

TOOLS USED

- HTML
- CSS
- JavaScript
- Validator.js (Library)

THEORY

Form validation ensures that user input is correct before submitting the form.

- **HTML** is used to create the structure
- **CSS** is used for styling
- **JavaScript** is used for validation logic
- **Validator.js** helps validate email and phone formats

Validation improves:

- ✓ Data accuracy
- ✓ User experience
- ✓ Security

PROCEDURE

◇ Step 1: Creating HTML Form

Explanation

An HTML form is created with:

- Email input field
- Phone number input field
- Error message display area
- Submit button

Code (HTML)

```
<!DOCTYPE html>

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Form Validation Experiment</title>
  <link rel="stylesheet" href="style.css">
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/validator/13.9.0/validator.min.js"></script>
</head>
<body>
  <div class="container">
    <h2>Contact Information</h2>
    <form id="contactForm">
      <div class="form-group">
        <label for="email">Email Address</label>
        <input type="email" id="email" name="email" placeholder="Enter your email">
        <div class="error-message" id="emailError"></div>
      </div>

      <div class="form-group">
        <label for="phone">Phone Number</label>
        <input type="tel" id="phone" name="phone" placeholder="Enter your phone
number">
        <div class="error-message" id="phoneError"></div>
      </div>

      <button type="submit">Submit</button>
    </form>
    <div id="successMessage" class="success-message hidden">Form submitted
successfully!</div>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

Output Screen (Step 1)

A simple form with:

- Email field
 - Phone field
 - Submit button
- (No validation yet)

The image shows a contact form titled "Contact Information". It is centered on a light gray background. The form consists of two input fields: "Email Address" and "Phone Number", each with a placeholder text "Enter your email" and "Enter your phone number" respectively. Below these fields is a blue "Submit" button. The entire form is enclosed in a light gray border with rounded corners.

◇ Step 2: Styling with CSS

Explanation

CSS is used to:

- Center the form
- Add colors and spacing
- Highlight input fields
- Show error styles

Features Added

- Rounded input boxes
- Button hover effect
- Error highlight (red border)
- Clean UI design

Code (CSS)

```
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600&display=swap');

body {
  font-family: 'Inter', sans-serif;
  background-color: #f0f2f5;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
  margin: 0;
}
```

```
.container {
  background-color: #ffffff;
  padding: 40px;
  border-radius: 12px;
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.1);
  width: 100%;
  max-width: 350px;
  transition: transform 0.3s ease;
}

.container:hover {
  transform: translateY(-5px);
}

h2 {
  color: #333;
  margin-top: 0;
  margin-bottom: 24px;
  text-align: center;
  font-weight: 600;
}

.form-group {
  margin-bottom: 20px;
}

label {
  display: block;
  margin-bottom: 8px;
  color: #555;
  font-weight: 500;
  font-size: 14px;
}

input {
  width: 100%;
  padding: 12px 16px;
  border: 1px solid #ccc;
  border-radius: 8px;
  font-size: 16px;
  box-sizing: border-box;
  transition: border-color 0.3s ease, box-shadow 0.3s ease;
  font-family: 'Inter', sans-serif;
}

input:focus {
  outline: none;
  border-color: #4a90e2;
  box-shadow: 0 0 0 3px rgba(74, 144, 226, 0.2);
}
```

```
}

input.error {
  border-color: #e74c3c;
  box-shadow: 0 0 0 3px rgba(231, 76, 60, 0.2);
}

.error-message {
  color: #e74c3c;
  font-size: 12px;
  margin-top: 6px;
  min-height: 15px;
  font-weight: 500;
}

button {
  width: 100%;
  padding: 14px;
  background-color: #4a90e2;
  color: white;
  border: none;
  border-radius: 8px;
  font-size: 16px;
  font-weight: 600;
  cursor: pointer;
  transition: background-color 0.3s ease, transform 0.1s ease;
  font-family: 'Inter', sans-serif;
  margin-top: 10px;
}

button:hover {
  background-color: #357abd;
}

button:active {
  transform: scale(0.98);
}

.success-message {
  margin-top: 20px;
  padding: 12px;
  background-color: #d4edda;
  color: #155724;
  border: 1px solid #c3e6cb;
  border-radius: 8px;
  text-align: center;
  font-weight: 500;
}
```

```
.hidden {  
  display: none;  
}
```

Output Screen (Step 2)

Styled form:

- Centered layout
- Blue button
- Clean design

Step 3: JavaScript Validation

Explanation

JavaScript is used to:

- Validate email using `validator.isEmail()`
- Validate phone using `validator.isMobilePhone()`
- Show error messages
- Show success message

Validation Types

Real-time validation (while typing)

Submit validation

Code (JavaScript)

```
document.addEventListener('DOMContentLoaded', () => {  
  
  const form = document.getElementById('contactForm');  
  const emailInput = document.getElementById('email');  
  const phoneInput = document.getElementById('phone');  
  const emailError = document.getElementById('emailError');  
  const phoneError = document.getElementById('phoneError');  
  const successMessage = document.getElementById('successMessage');  
  
  // Function to show error  
  const showError = (input, errorElement, message) => {  
    input.classList.add('error');  
    errorElement.textContent = message;  
  };  
  
  // Function to clear error  
  const clearError = (input, errorElement) => {  
    input.classList.remove('error');  
    errorElement.textContent = '';  
  };  
});
```

```
// Real-time validation for email
emailInput.addEventListener('input', () => {
  if (emailInput.value.trim() === '') {
    clearError(emailInput, emailError);
  } else if (!validator.isEmail(emailInput.value)) {
    showError(emailInput, emailError, 'Please enter a valid email address.');
```

```
  } else {
    clearError(emailInput, emailError);
  }
});

// Real-time validation for phone
phoneInput.addEventListener('input', () => {
  const phoneVal = phoneInput.value.trim();
  if (phoneVal === '') {
    clearError(phoneInput, phoneError);
  } else if (!validator.isMobilePhone(phoneVal, 'any')) {
    showError(phoneInput, phoneError, 'Please enter a valid phone number.');
```

```
  } else {
    clearError(phoneInput, phoneError);
  }
});

// Form submission validation
form.addEventListener('submit', (e) => {
  e.preventDefault();

  let isValid = true;
  const emailVal = emailInput.value.trim();
  const phoneVal = phoneInput.value.trim();

  // Validate Email
  if (emailVal === '') {
    showError(emailInput, emailError, 'Email is required.');
```

```
    isValid = false;
  } else if (!validator.isEmail(emailVal)) {
    showError(emailInput, emailError, 'Please enter a valid email address.');
```

```
    isValid = false;
  }

  // Validate Phone
  if (phoneVal === '') {
    showError(phoneInput, phoneError, 'Phone number is required.');
```

```
    isValid = false;
  } else if (!validator.isMobilePhone(phoneVal, 'any')) {
    showError(phoneInput, phoneError, 'Please enter a valid phone number format.');
```

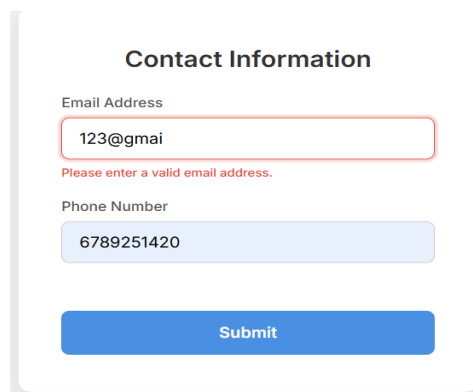
```
    isValid = false;
  }
});
```

```
// Output success message
if (isValid) {
    successMessage.classList.remove('hidden');
    form.reset();

    setTimeout(() => {
        successMessage.classList.add('hidden');
    }, 3000);
} else {
    successMessage.classList.add('hidden');
}
});
});
```

Output Screens (Step 3)

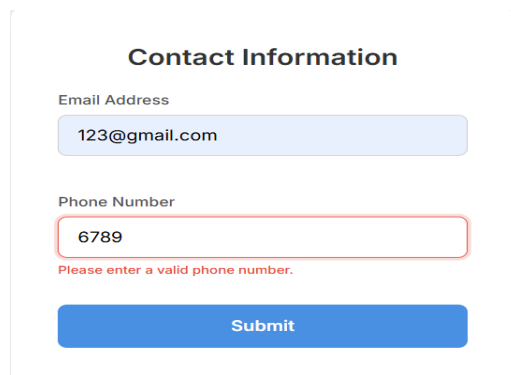
Case 1: Invalid Email



The screenshot shows a contact form titled "Contact Information". It has two input fields: "Email Address" and "Phone Number". The "Email Address" field contains "123@gmai" and is highlighted with a red border. Below it, a red error message reads "Please enter a valid email address." The "Phone Number" field contains "6789251420" and has a light blue background. At the bottom is a blue "Submit" button.

Shows: "Please enter a valid email address"

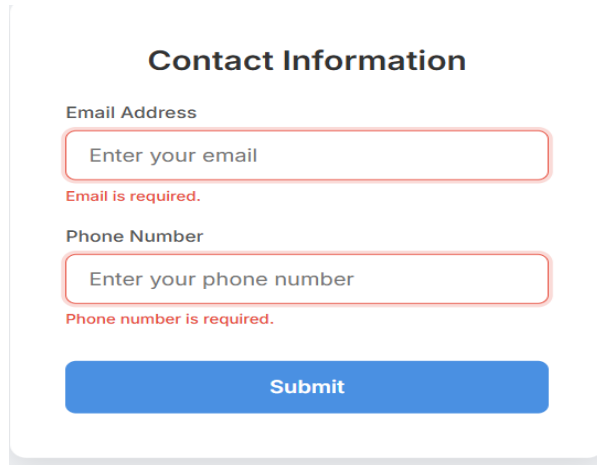
Case 2: Invalid Phone



The screenshot shows the same contact form titled "Contact Information". The "Email Address" field now contains "123@gmail.com" and has a light blue background. The "Phone Number" field contains "6789" and is highlighted with a red border. Below it, a red error message reads "Please enter a valid phone number." At the bottom is a blue "Submit" button.

Shows: "Please enter a valid phone number"

Case 3: Empty Fields



Contact Information

Email Address

Email is required.

Phone Number

Phone number is required.

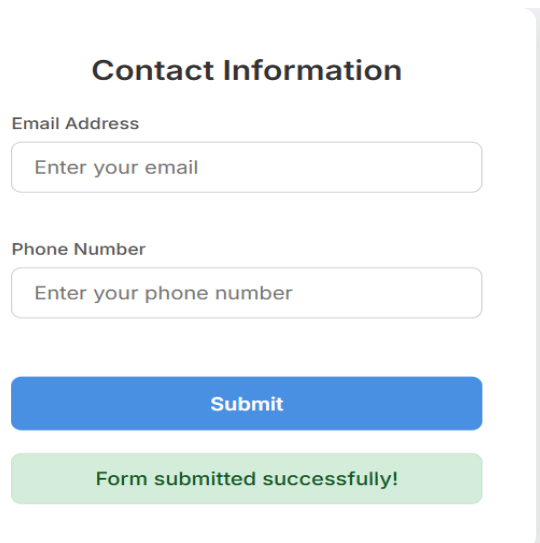
Submit

This screenshot shows a web form titled "Contact Information". It contains two input fields: "Email Address" and "Phone Number". Both fields are empty and have a red border, indicating they are required. Below each field is a red error message: "Email is required." and "Phone number is required." respectively. At the bottom of the form is a blue "Submit" button.

Shows:

Email is Required
Phone number is required

Case 4: Valid Input



Contact Information

Email Address

Phone Number

Submit

Form submitted successfully!

This screenshot shows the same "Contact Information" form as in Case 3, but with valid input. The "Email Address" and "Phone Number" fields now contain the placeholder text "Enter your email" and "Enter your phone number" respectively. The "Submit" button is still present. Below the button, a green message box displays "Form submitted successfully!".

Shows:

Form submitted successfully